ICT Trading Bot 2026 — Architecture, SOP & Runbook

Oct 17, 2026 · @Prabhu

1. Executive summary

We built a rule-based intraday options trading bot. It runs unattended on Google Cloud, takes market data from Schwab and places orders through Alpaca. Its live strategy is a **first 5-minute candle breakout on SPY/QQQ** using short-dated (0DTE/1DTE) options. Everything runs from one Docker image: a Streamlit dashboard plus six scheduled Cloud Run jobs.

What it does each trading day

- **08:45 ET:** a pre-market plan is emailed. It covers Schwab login health, FOMC/CPI/NFP events, earnings, shock news, prior-day levels, gaps and GEX walls.
- **09:30–15:45 ET:** the engine checks exits every 30 s and scans for entries at each 5-minute bar close. Risk gates run before every order.
- **15:45 ET:** all positions are flattened. Options are never held overnight.
- **18:00–18:15 ET:** nightly jobs archive GEX, update a ~650 MB market-data database and re-run the strategy backtest. That backtest drives the live health monitor.

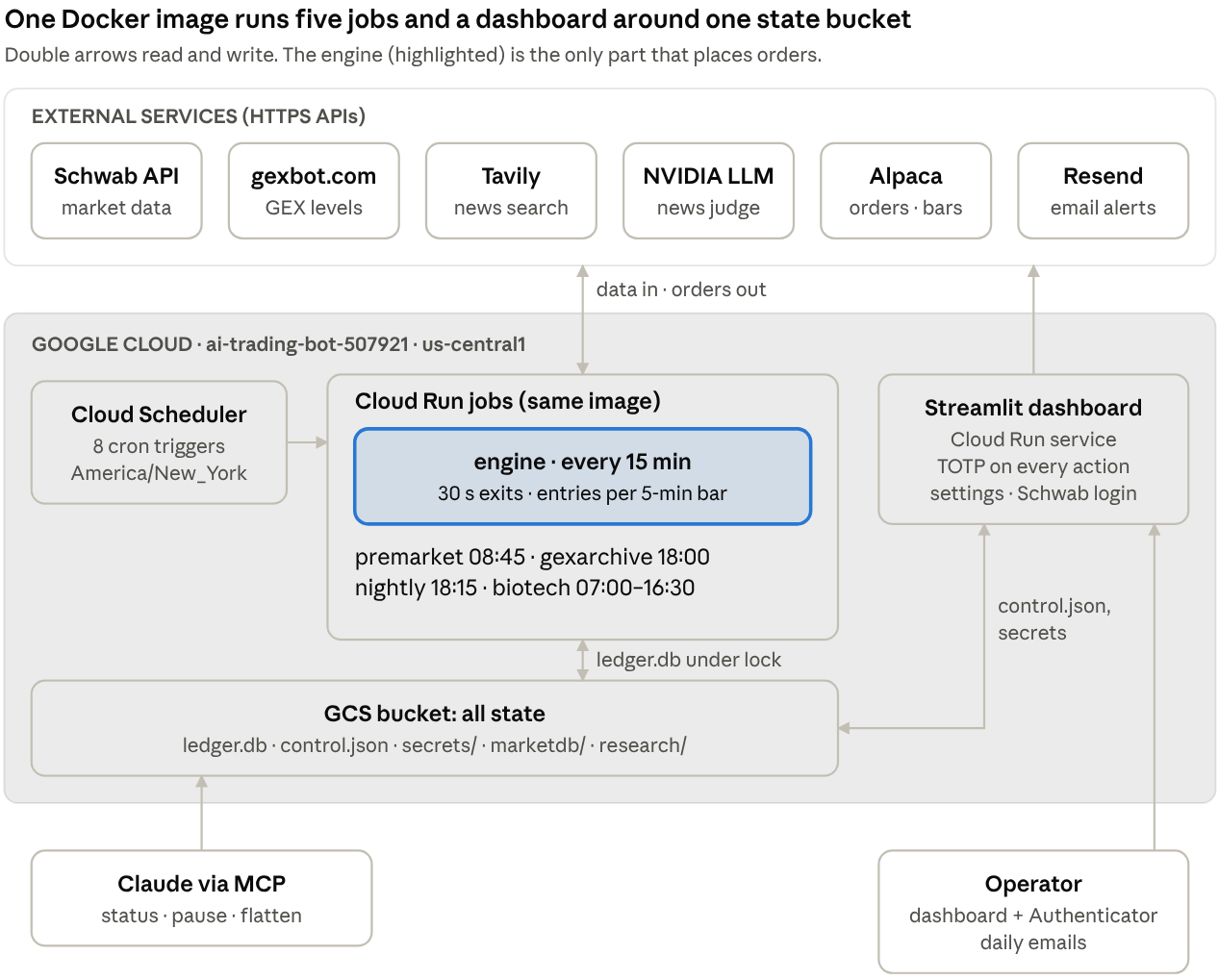
Where it stands (Oct 17, 2026)

Item	Status
Broker	Alpaca **paper** (live requires both a dashboard switch and `ALLOW_LIVE_TRADING=true`)
Live strategy	`first_candle` / `smart` (gap-graded SPY/QQQ, 0DTE when the gap agrees, else 1DTE)
Backtest, Jan 2–Jun 3 2026 from $10k	ends ~$21.3k (real option prices, base sizing)
Backtest, Jun 4–Oct 9 2026 from $10k	ends ~$21.0k
Paper-only research tracks	GEX ideas tab, biotech gap scanner, FOMC reaction trade, LLM news judge (shadow)

Item	Status
Code	37 commits since Sep 22 2026, ~15k lines Python, 17 test files

The main risk is that the edge is recent. The same rule lost money in 2025 (about −$6.4k from $10k). For that reason, the bot carries a kill switch, a hard equity floor and a monitor that cuts size when recent trades lose. See section 8.

2. Architecture



system architecture · 6 external services, 5 jobs, 1 dashboard, 1 bucket

Scheduler wakes the jobs; every job and the dashboard share state only through the bucket. Schwab supplies data, Alpaca receives orders, and the operator and Claude can pause or flatten but never open trades directly.

3. Component catalogue

All code lives in `src/`. One CLI, `python -m src.app <command>`, is the entry point for every job.

Layer	Module(s)	Responsibility
Entry point	`src/app.py`	CLI commands: `premarket`, `run`, `cycle`, `flatten`, `scan`, `gex-archive`, `nightly`, `biotech`, `backtest`, `test-email`
Engine loop	`src/agents/orchestrator.py`	Every ~30 s: reconcile with the broker, manage exits, then scan, gate and enter at each 5-min bar; writes heartbeat
Agents	`market_analyst`, `risk_agent`, `execution_agent`, `monitor_agent`, `news_agent`, `premarket`	Signal scan; sizing and session/account gates; order placement and option selection; exit rules; Tavily news; morning plan
LLM judge	`src/agents/llm.py`	NVIDIA-hosted Nemotron reads headlines and returns block/caution/none. It can veto but never open a trade. Default mode is `shadow`
Strategies	`src/strategy/first_candle.py`, `vrz.py`, `gex.py`, `gex_plan.py`, `convergence.py`, `biotech_gap.py`, `fomc_react.py`, `indicators.py`	First-candle (live), VRZ sweep-and-reject, GEX levels and ideas, convergence scoring, paper-only side strategies, VWAP/profile/RVOL
Market data	`src/data/schwab.py`, `market.py`, `gexbot.py`	Schwab OAuth client (quotes, bars, chains, greeks); facade with Alpaca bar fallback; gexbot.com live GEX and EOD archive

Layer	Module(s)	Responsibility
Brokers	`src/brokers/base.py`, `alpaca.py`, `ibkr.py`, `platforms.py`	One `Broker` interface. Alpaca paper/live supported, IBKR experimental, Robinhood not available
State	`src/core/store.py`, `ledger.py`, `settings.py`, `clock.py`, `events.py`	GCS-mirrored files with a lock; SQLite ledger; bounded settings; market calendar; FOMC/CPI/NFP calendar
Research data	`src/market_db.py`, `src/nightly.py`, `src/backtest/`	~650 MB SQLite market DB (daily, 5-min, option bars, chains, GEX, GARCH); nightly update; event-driven backtester
UI	`src/dashboard.py`, `dashboard_pages.py`, `dashboard_gex.py`	Streamlit with TOTP login. Tabs: Today, Performance, Trades, Signals & levels, GEX ideas, How it works, Research, Backtest, Settings, Platforms, Logs
Alerts	`src/alerts/notifier.py`	HTML email via Resend, de-duplicated per key
AI access	`src/mcp_server.py`	MCP server with read tools plus pause and flatten. It cannot open trades
Tests	`tests/` (17 files)	Rules, strategy, engine with a fake broker, backtest, store, platforms, LLM parsing

State ownership prevents last-writer-wins corruption. Only engine jobs write `ledger.db`, and only while holding the engine lock. Only the dashboard writes `control.json`. Only the Schwab re-auth page writes `secrets/`.

4. Technology stack and why

The guiding choice was a single small Python codebase on serverless Google Cloud. It costs nothing when the market is closed and needs no servers to patch.

Technology	Used for	Why we chose it
Python 3.12 + pandas / numpy / scipy	All logic, indicators, Black-Scholes, backtests	One language for research and production, so a backtested rule ships unchanged
Schwab Trader API (data only)	Quotes, 5-min bars, option chains with greeks and open interest	Free real-time chains with OI, needed for our own GEX. Never used for orders
Alpaca (`alpaca-py`)	Order execution (paper and live), backup bars, historical SIP and option bars	Free paper account with the same API as live, so switching is a setting, not a rewrite
IBKR Client Portal API	Future broker (experimental)	Real options broker the user plans to move to; it sits behind the same `Broker` interface
gexbot.com API	Live GEX levels, daily EOD archive	Keeps GEX working when the Schwab login lapses, and builds a GEX history for backtests
Tavily	News search per ticker and market-wide	Simple search API that returns clean snippets for an agent
NVIDIA hosted LLM (Nemotron)	News judge (veto/shadow)	OpenAI-compatible API on a free tier. Kept advisory-only: it cannot add points or open trades
Resend	Email alerts and reports	One HTTPS call, no mail server
Streamlit + Plotly	Dashboard and charts	Fast to build a Python-only UI with interactive charts
pyotp (Google Authenticator)	Login and every action on the dashboard	The service is public on Cloud Run, so a TOTP gate protects it without extra infrastructure

Technology	Used for	Why we chose it
SQLite	Trade ledger; market-data DB	Zero-ops, single file, easy to copy to and from the bucket
Google Cloud Storage	State: ledger, control file, tokens, secrets, research output, market DB	Durable, cheap and shared by every job; a generation-matched lock prevents two writers
Cloud Run service	Dashboard (1 instance, 4 GiB, session affinity)	Serverless HTTPS with a fixed URL, which Schwab OAuth needs as its redirect URI
Cloud Run Jobs	premarket, engine, gexarchive, nightly, biotech	Run-to-completion batch work; billed only while running
Cloud Scheduler	Cron triggers in America/New_York	Time-zone aware schedules that call the jobs directly
Cloud Build + Artifact Registry	Build and store the image	One `gcloud builds submit`; optional CI config runs the tests first
Docker (python:3.12-slim)	One image for service and all jobs	Each job only overrides the command, so everything runs the same code version
MCP (`mcp` SDK)	Let Claude read status, pause, flatten	Natural-language monitoring without giving an AI the power to open trades
pytest	17 test files	Engine tested end-to-end against a fake broker before any deploy

5. Schedules and data flow

Eight Cloud Scheduler triggers run five Cloud Run jobs on weekdays (America/New_York). Market holidays and half days are handled in code by `src/core/clock.py`.

Trigger	Time (ET, Mon–Fri)	Job	Command	What it does
`ict-biotech-scan` / `-scan8`	07:00, 07:30, 08:00	`ai-trading-biotech`	`biotech`	Paper-only pre-market biotech gap scan
`ict-premarket`	08:45	`ai-trading-premarket`	`premarket`	Morning plan: Schwab token health, events, earnings, news, levels, GEX → email + dashboard
`ict-engine-open`	09:30, 09:45	`ai-trading-engine`	`run --minutes 14`	Engine loop for the opening window
`ict-engine-day`	every 15 min, 10:00–15:45	`ai-trading-engine`	`run --minutes 14`	Engine loop: exits every 30 s, entries per 5-min bar, flatten at 15:45
`ict-biotech-settle`	16:30	`ai-trading-biotech`	`biotech`	Settles paper biotech trades
`ict-gex-archive`	18:00	`ai-trading-gexarchive`	`gex-archive`	Saves gexbot EOD snapshots to the bucket
`ict-nightly`	18:15	`ai-trading-nightly`	`nightly`	Updates market DB, saves EOD chains, prunes ledger, replays GEX ideas, re-runs the smart-rule backtest

Trading-day rules for the live first-candle strategy

- No entries in the first 15 minutes or within ±30 minutes of a scheduled event. FOMC blackout runs 13:30–15:00.
- No trades on FOMC decision or CPI days, nor the session before or after an FOMC decision.
- Entry must break on the 09:35–09:45 bars. Target is 3× the first candle's range. Break-even after 1×. At 11:00, exit if it has not moved 0.5×.

- Last entry 15:00. Flatten 15:45, or 12:45 on half days.

Where data lives

Store	Contents	Writer
`gs://ai-trading-ledger-bucket-464783405434/ledger.db`	Trades, signals, levels, GEX levels, alerts, kv status, run log	Engine and nightly, under lock
`.../control.json`	Pause, flatten request, settings, broker choice	Dashboard only
`.../secrets/`	Schwab refresh token, broker credentials	Dashboard only
`.../marketdb/market_data.db`	Daily and 5-min bars, option bars, chains, events, GARCH, GEX history	Nightly job
`.../research/`, `gexbot/eod/`, `backtests/`	Smart-rule results, GEX replay, gexbot archive, backtest output	Nightly and gexarchive jobs

6. Standard operating procedures

The bot runs itself. The operator's job is five minutes before the open, a glance at the close, and a weekly review.

6.1 Daily checklist

When (ET)	Action	Where	Pass condition
08:50	Read the pre-market email	Inbox	Email arrived; Schwab token has more than 1.5 days left; no event blackout you didn't expect
09:00	Open dashboard, enter Authenticator code	Today tab	Broker shows **paper** (or intended live); strategy shows `first_candle / smart`; not paused

When (ET)	Action	Where	Pass condition
09:50	Check entry	Today and Trades tabs	Either one trade with stop/target set, or a logged reason for no trade
15:50	Confirm flat	Today tab	No open bot positions
18:45	Check nightly	Inbox	No "Nightly database update needs attention" email

6.2 Weekly checklist

- **Every 7 days: renew the Schwab login.** Dashboard → Settings → Schwab connection → log in → enter Authenticator code. Jobs pick it up on the next run.
- **Friday evening:** read the GEX ideas weekly replay email.
- **Weekly:** review Performance and Research tabs. Note whether monitor C has moved the profile to `safer` or `defensive`.
- **Monthly:** confirm the event calendar (FOMC, CPI, NFP) covers the next month.

6.3 Risk rules (enforced in code)

Rule	Setting	Value
Starting capital	trading capital	$10,000, profits reinvested
Premium per trade, phase 1	`fc_size_scale` × grade	2× (10–14% of equity)
Phase 2 once equity first hits $50k	`fc_phase2_scale`	0.5× phase 1 (5–7%)
Hard floor	`fc_floor_equity` / `fc_phase2_floor`	No trades below $5k (phase 1) or $35k (phase 2)
Half size	`fc_halfsize_lookback`	While the last 10 trades average a loss
Kill switch	`fc_kill_*`	Win rate < 25% over 20 trades, or 15% drawdown

Rule	Setting	Value
Live money	dashboard + env	Needs both the dashboard choice **and** `ALLOW_LIVE_TRADING=true` on the job
Data outage	always	Fails closed: no new entries if Schwab data is down; exits still run

6.4 Incident procedures

Symptom	Likely cause	Action
Email: Schwab login expiring or failed	7-day refresh token lapsed	Re-auth in Settings (6.2). GEX falls back to gexbot meanwhile
No trades and "market data unavailable" in Logs	Schwab outage or token	Re-auth; check Logs tab. Bot is safe: it fails closed
Position open after 15:45	Engine job failed or order rejected	Dashboard → Flatten, or MCP `request_flatten`; then close manually in Alpaca if still open
Unexpected behaviour mid-day	Any	Dashboard → Pause (exits keep running). Investigate in Logs and Cloud Run job logs
Nightly failure email	Data source or memory	Read the JSON in the email; rerun `gcloud run jobs execute ai-trading-nightly`
"Lock held" errors	Two writers overlapping	Wait one cycle; the lock expires on its TTL
Switching broker	Planned change	Platforms tab. The switch waits until open trades close

6.5 Change management

1. Work on a branch; run `venv/bin/python -m pytest -q tests`.
2. Backtest the change; report Jan 2–Jun 3 and Jun 4–today 2026 separately, each from $10k.

3. Commit. `deploy/deploy.sh` refuses to ship uncommitted source files.
4. Run `deploy/deploy.sh` (and `deploy/scheduler.sh` if schedules changed) outside market hours.
5. Watch the next premarket email and the first engine run.

7. Runbook: build it from scratch

A fresh build takes about half a day, most of it waiting on account approvals. Follow the phases in order; each ends with a check.

Phase 0 — Accounts and keys

Service	What to get	Env var(s)
Google Cloud	Project with billing; `gcloud` CLI logged in	(project id in `deploy/*.sh`)
Schwab Developer	App with Market Data API; callback = the dashboard URL (set after Phase 4)	`SCHWAB_CLIENT_ID`, `SCHWAB_CLIENT_SECRET`, `SCHWAB_REDIRECT_URI`
Alpaca	Paper keys (live keys optional)	`APCA_API_KEY_ID`, `APCA_API_SECRET_KEY`, `ALPACA_DATA_FEED`, `APCA_LIVE_*`
gexbot.com	Classic plan key	`GEXBOT_API_KEY`
Tavily	API key	`TAVILY_API_KEY`
NVIDIA build	API key, model id	`NVIDIA_API_KEY`, `NVIDIA_MODEL`
Resend	API key, destination address	`RESEND_API_KEY`, `ALERT_EMAIL_TO`
Authenticator	Base32 secret (`python -c "import pyotp;print(pyotp.random_base32())"`), added to Google Authenticator	`MFA_SECRET`

Phase 1 — Local setup

```
git clone <repo> ict-trading-bot-2026 && cd ict-trading-bot-2026
python3.12 -m venv venv && venv/bin/pip install -r requirements-dev.txt
venv/bin/python -m pytest -q tests          # all tests must pass
```

Check: tests green. Without `GCS_BUCKET_NAME`, state is kept in `./data`.

Phase 2 — Market-data database (for backtests and the nightly job)

```
venv/bin/python src/market_db.py all       # builds
data/marketdb/market_data.db (~650 MB)
venv/bin/python -m src.app backtest --tickers SPY,QQQ --start 2026-01-02 --
end 2026-06-03
```

Check: the backtest prints trades and an equity curve. Leave ~1.5 GB free disk.

Phase 3 — Google Cloud foundations

```
PROJECT=<your-project>; REGION=us-central1
gcloud config set project $PROJECT
gcloud services enable run.googleapis.com cloudbuild.googleapis.com \
  artifactregistry.googleapis.com cloudscheduler.googleapis.com
storage.googleapis.com
gcloud artifacts repositories create cloud-run-source-deploy --repository-
format=docker --location=$REGION
gcloud storage buckets create gs://<bucket> --location=$REGION --uniform-
bucket-level-access
gcloud storage cp data/marketdb/market_data.db
gs://<bucket>/marketdb/market_data.db
```

Give the default compute service account Storage Object Admin on the bucket, and the Cloud Run Invoker role (Scheduler uses it). Check: `gcloud storage ls gs://<bucket>/marketdb/` lists the DB.

Phase 4 — Configure and deploy

1. `cp deploy/env.example.yaml env.yaml` and fill every value. Set `GCS_BUCKET_NAME`. Keep `ALLOW_LIVE_TRADING: "false"`. Never commit this file.
2. Edit `PROJECT` at the top of `deploy/deploy.sh` and `deploy/scheduler.sh` (or export `PROJECT=`).

3. Commit everything, then run `deploy/deploy.sh`. It builds the image and deploys the dashboard plus five jobs.
4. Copy the dashboard URL from the output. Set it as `SCHWAB_REDIRECT_URI` in `env.yaml` **and** as the callback in the Schwab developer app. Redeploy.
5. Run `deploy/scheduler.sh` to create the eight triggers.

Check: `gcloud run jobs list` shows 5 jobs; `gcloud scheduler jobs list --location=$REGION` shows 8 enabled triggers.

Phase 5 — First login and smoke test

1. Open the dashboard URL; log in with the Authenticator code.
2. Settings → Schwab connection → log in to Schwab → enter code. Token is saved to `secrets/`.
3. Platforms → choose **Alpaca paper**. Settings → Strategy → `first_candle`, mode `smart`.
4. `gcloud run jobs execute ai-trading-premarket --region=$REGION --wait`, then confirm the email arrives (or run `python -m src.app test-email` locally).
5. Signals & levels → **Run post-open scan now** during market hours.

Check: premarket email received; scan shows levels; Logs tab shows the engine heartbeat.

Phase 6 — Paper run, then live

- [ ] Run on paper for at least 20 trading days and compare with the nightly backtest.
- [ ] Confirm daily flatten, kill switch and floor behaviour in the ledger.
- [ ] Only then: enter live keys, set `ALLOW_LIVE_TRADING: "true"`, redeploy, and choose Alpaca live in Platforms.

Optional — MCP access from Claude

Add to the Claude MCP config: command `venv/bin/python`, args `-m src.mcp_server`, cwd = the repo, env `GCS_BUCKET_NAME=<bucket>`. Tools: status, trades, levels, events, analyze, news, pause, flatten.

8. Known limitations, risks and next steps

The biggest risk is the strategy, not the software: its 2026 edge has not held in other years.

Area	Limitation or risk	Mitigation today
Strategy	Smart rule lost ~$6.4k from $10k in 2025; several rules were chosen with 2026 data (in-sample)	Paper first; kill switch; half-size rule; monitor C; equity floor
Sizing	At 2× sizing, Monte Carlo shows ~3% chance of falling to $3k within a year, median drawdown about −50%	Phase-2 cut to half size at $50k; $5k floor
Schwab login	Refresh token expires every 7 days; manual re-auth	Warning email at 1.5 days; gexbot fallback for GEX; fails closed
Single dashboard instance	Streamlit sessions need one instance; dashboard is public behind TOTP only	TOTP on login and every action; secrets kept in a private bucket
Event calendar	FOMC/CPI/NFP dates must be kept current	Nightly loads official dates into the market DB on Mondays
Untested paths	IBKR adapter never run against a real gateway; LLM judge and GEX ideas not backtestable	IBKR marked experimental; LLM in shadow; GEX ideas paper-tracked
Repo hygiene	~20 one-off `backtest_*.py` research scripts in `src/`; `legacy/` folder	Not imported by jobs; candidates for a `research/` move

Next steps

- [ ] Finish 20+ paper trading days and compare live fills with the nightly backtest.
- [ ] Decide on `news_llm_mode = veto` after reviewing shadow verdicts against outcomes.
- [ ] Decide whether GEX ideas graduate from paper after a few months of replay data.
- [ ] Test the IBKR adapter on a paper (DU…) account before any broker move.
- [ ] Move research scripts out of `src/`.